

A Artifact Appendix

A.1 Abstract

The artifact contains the Python optimizers (LITE, LINE, EZR, RANDOM wrappers plus DEHB, SMAC and TPE via their public packages), a Lua scoring harness, pre-computed results for all optimizers over the MOOT benchmark tasks, and scripts that rebuild the paper’s summary table (Table 3, percent-best and Δ per budget), Figure 4 (percent-best vs. label budget) and Figure 5 (runtime box plots). The cheap path (table and figures from pre-computed results) runs in seconds. The expensive path (re-running all optimizers on all tasks) is optional.

A.2 Description & Requirements

A.2.1 How to access

<https://github.com/KKGanguly/NEO>, commit 5a0d487 (2026-07-27), MIT license, ≈ 104 MB cloned (mostly pre-computed results). The benchmark data is *not* in this repository: clone <https://github.com/timm/moot> into `NEO/moot/` (the Makefiles and Lua harness expect `NEO/moot/optimize/**/*.csv`; the README’s `data/moot/path` is wrong).

A.2.2 Hardware dependencies

Authors’ rig: 4-core 1.30 GHz Linux, 16 GB RAM, no GPU. E1–E3 below run on any laptop. E4 (full re-run) benefits from many cores; the generated `commands.sh` launches one process per dataset in parallel.

A.2.3 Software dependencies

Python (README says 3.13; `.python-version` says 3.11; 3.13 confirmed for E2–E3) with `requirements.txt` (pandas, matplotlib, seaborn, scipy, numba, ConfigSpace, dehb, smac, optuna, optunahub, hebo; CMake/SWIG needed to build SMAC). Lua 5.x, GNU make, bash, awk, egrep, gawk. Lua is required by `make comparez` and is *not* mentioned in the README. No container is provided.

A.2.4 Benchmarks

127 multi-objective SE optimization tasks from MOOT (Table 1 of the paper): software configuration, HPO, product lines, project health, cost/effort, defect prediction. Note: the committed `report.csv` covers 112 tasks; 15 tasks named in paper v2 are absent from the precomputed results.

A.3 Set-up

```
git clone https://github.com/KKGanguly/NEO
cd NEO
git clone https://github.com/timm/moot # NEO/moot
python3 -m venv .venv
. .venv/bin/activate
```

```
pip install -r requirements.txt
brew install lua # or: apt install lua5.4
```

```
Basic test (no data needed): cd
experiments/LUA_run_all && make report must
print nothing and rewrite results/optimization_
performance/report.csv.
```

A.4 Evaluation workflow

A.4.1 Major Claims

- (*C1*) With label budgets of 6–200, simple pool-based optimizers (LITE, LINE, RANDOM, EZR) match or beat SMAC, TPE and DEHB in percent-best rank and normalized improvement Δ ; LITE is best overall and DEHB worst. Shown in Table 3 (§5.5). Reproduced by E1.
- (*C2*) Percent-best gains flatten after ≈ 100 labels; small budgets favor LINE/RANDOM, mid budgets favor LITE. Shown in Figure 4. Reproduced by E2.
- (*C3*) RANDOM runs $>10,000\times$ faster than the membership-query optimizers; SMAC is >2 orders of magnitude slower than LITE/EZR. Shown in Figure 5. Reproduced by E3.
- (*C4*) The BINGO effect (§6, Table 4, Figure 7: data collapses into few occupied buckets). **Not covered:** no script in the repository regenerates Table 4 or Figure 7.

A.4.2 Experiments

Experiment (E1): Summary table [5 human-min + 1 compute-sec from committed scores; Lua rescoring untimed]. Rebuilds Table 3.

[*Preparation*] Set-up above. `comparez.out` (Lua percent-best/ Δ per task) is committed, so the Lua step is optional.

[*Execution*] `cd experiments/LUA_run_all; make report`. To rescore from the raw runs: `make comparez` first (runs `lua run_all.lua --comparez` once per MOOT csv, reading `results/results_*/` through `FileResultsReader.py`; RANDOM and LINE are computed inside the Lua script, not from Python runs).

[*Results*] `results/optimization_performance/report.csv`: one row per task, columns `<OPT>-<budget>` and `<OPT>-<budget>.sd`; last row is the column mean. Compare the mean row with Table 3 (e.g. LITE = ACT column: 32, 54, 70, 75, 82, 88, 94; DEHB: 24, 25, 31, 39, 59, 64, 69). Regenerating from the committed `comparez.out` gave ± 1 differences in a few cells of the mean row vs. the committed csv.

Experiment (E2): Figure 4 [2 human-min + 1 compute-sec].

[*Execution*] `cd experiments; MPLBACKEND=Agg`

python3 optim_performance_comp.py. Without
MPLBACKEND=Agg the script blocks on plt.show().
[Results] results/optimization_performance/
optimization_performance_comparison.png: percent-
best vs. budget, one line per optimizer; LITE on top from
budget 12 onward, DEHB lowest everywhere (C2).

Experiment (E3): Figure 5 [2 human-min + 1 compute-
sec].

[Execution] cd experiments; MPLBACKEND=Agg
python3 performance_box.py (reads report_tmp.csv,
written by E1).

[Results] results/runtime_plot/families_runtime_
boxplot.png, log-ms boxes for DEHB, SMAC, TPE,
LITE, EZR, LINE, RANDOM; RANDOM lowest, SMAC
highest (C3).

Experiment (E4, optional): Re-run optimizers [10
human-min + days of compute, not measured]. 127 tasks
× 7 budgets × 20 repeats per optimizer.

[Execution] From repo root: make generate-commands
NAME=SMAC (also DEHB, TPE, Active_Learning, EZR),
then cd experiments && ./commands.sh. Output
replaces results/results_<NAME>/<NAME>/<task>
_<budget>.csv (three lines: sampled configs, d2h scores,
seconds, one entry per repeat). Then E1–E3. Stochastic;
expect Table 3 cells to move by a few points.

A.5 Notes on Reusability

New task: drop a MOOT-format csv (goal columns
end in +/-) under moot/optimize/<group>/; make
generate-commands picks it up; add its name to
the comparez glob in experiments/LUA_run_all/
Makefile. New optimizer: subclass optimizers/base_
optimizer.py, register in experiments/experiment_
runner_cluster.py, add a results reader block
in run_all.lua. Budgets and repeats are the
--budget/--repeats flags in the root Makefile.

A.6 General Notes

Known gaps an evaluator will hit: (1) README points to
data/moot/, code expects moot/; (2) README calls the
summary "Table 4", paper v2 numbers it Table 3; (3) Lua
and gawk are undocumented dependencies; (4) 112 of 127
tasks have precomputed results; (5) no Docker image; (6)
plt.show() in E2 needs a headless backend on servers.